



UEA
UNIVERSIDADE
DO ESTADO DO
AMAZONAS



RELATÓRIO TÉCNICO | 1º BIMESTRE

EcoOrigem

Rastreabilidade de produtos da bioeconomia amazônica

Blockchain local em Python com contrato inteligente, interface web e execução em Docker

INSTITUIÇÃO

Universidade do Estado do Amazonas

DISCIPLINA

Oficina de Desenvolvimento de
Sistemas III

PROFESSOR

Prof. Dr. Fábio Santos

DATA

Setembro de 2026

EQUIPE

Ana Beatriz Maciel Nunes

Fernando Luiz da Silva Freire

Juliana Ballin Lima

ATIVIDADE

Trabalho do 1º bimestre: aplicação com
blockchain local

Sumário

1 Introdução.....	3
2 Problema e objetivos.....	3
2.1 Problema.....	3
2.2 Objetivo geral.....	3
2.3 Objetivos específicos.....	3
2.4 Escopo.....	4
3 Justificativa do uso de blockchain.....	4
4 Arquitetura.....	5
5 Implementação.....	6
5.1 Blockchain.....	6
5.2 Contrato inteligente.....	7
5.3 Nó da blockchain.....	9
5.4 Interface web.....	9
5.5 Infraestrutura.....	10
6 Dados na blockchain.....	10
7 Testes.....	11
7.1 Estratégia.....	11
7.2 Cenários de operações válidas, inválidas e de integridade.....	12
7.3 Resultados obtidos.....	12
8 Como executar.....	13
8.1 Com Docker.....	13
8.2 Sem Docker.....	13
9 Limitações.....	13
10 Conclusão.....	14
Referências.....	14
Apêndice A - Cenários de teste.....	15
Apêndice B - Evidências da interface.....	17

1 Introdução

A bioeconomia amazônica movimenta produtos como açaí, castanha-do-brasil, murumuru, cupuaçu, andiroba e óleos vegetais. Entre a comunidade extrativista e o consumidor final, cada lote passa por beneficiamento, transporte e distribuição, e cada etapa fica sob responsabilidade de uma organização diferente. As informações sobre essa trajetória costumam ficar dispersas em planilhas, notas e sistemas isolados, sem garantia de que não foram alteradas depois.

Este relatório apresenta o EcoOrigem, um protótipo de rastreabilidade que registra a trajetória de cada lote em uma blockchain local implementada em Python. O trabalho foi desenvolvido para a disciplina Oficina de Desenvolvimento de Sistemas III e segue a lógica apresentada em aula: bloco com hash SHA-256, encadeamento por hash anterior, prova de trabalho com nonce, validação da cadeia e uma aplicação descentralizada formada por interface, contrato inteligente e blockchain.

2 Problema e objetivos

2.1 Problema

Como garantir um histórico confiável e rastreável da origem e das movimentações de produtos da bioeconomia amazônica, de modo que cada participante da cadeia consiga comprovar o que registrou e ninguém consiga alterar o passado sem deixar rastro?

Hoje o histórico de um lote depende de registros mantidos separadamente por produtor, beneficiador, transportador e distribuidor. Qualquer um deles pode corrigir, apagar ou reescrever a própria versão, e o consumidor não tem como verificar qual versão é a verdadeira. Isso enfraquece a confiança na origem declarada e dificulta certificações e auditorias.

2.2 Objetivo geral

Desenvolver uma aplicação sobre uma blockchain local em Python que registre, consulte e audite o ciclo de vida de lotes de produtos da bioeconomia amazônica, com regras de negócio executadas por um contrato inteligente.

2.3 Objetivos específicos

- Implementar uma blockchain local com blocos, hash SHA-256, prova de trabalho, árvore de Merkle e validação da cadeia.
- Implementar um contrato inteligente com perfis de acesso, ordem obrigatória das etapas e validação das entradas.
- Disponibilizar uma interface web para registrar operações, consultar lotes e visualizar a blockchain.

- Demonstrar que operações inválidas ou sem permissão são rejeitadas e que a adulteração de dados gravados é detectada.
- Documentar quais dados ficam na blockchain, testar operações válidas e inválidas e empacotar a solução para execução reproduzível.

2.4 Escopo

O protótipo cobre o registro de lotes, o beneficiamento, o transporte, a confirmação de recebimento, a finalização e o anexo de documentos por hash. Ficam fora do escopo a integração com sensores, certificadoras e sistemas reais de cooperativas, a rede blockchain pública e o uso de moeda ou tokens.

3 Justificativa do uso de blockchain

A rastreabilidade de uma cadeia produtiva reúne três características que favorecem a blockchain: há vários participantes independentes, nenhum deles deve poder reescrever o histórico sozinho e todos precisam verificar os mesmos registros. A Tabela 1 compara a solução com um banco de dados relacional tradicional operado por uma única organização.

Tabela 1 - Banco de dados tradicional e blockchain no problema do EcoOrigem

Critério	Banco de dados centralizado	Blockchain do EcoOrigem
Quem controla o histórico	O administrador do banco pode alterar ou apagar registros	Nenhum participante altera um bloco sem quebrar o encadeamento
Deteção de fraude	Depende de logs que o mesmo administrador controla	Qualquer nó recalcula os hashes e a raiz de Merkle e detecta a alteração
Regras de negócio	Ficam no código da aplicação, que pode ser contornado	Ficam no contrato inteligente, executado sobre a cadeia
Autoria das operações	Usuário e senha em tabela do sistema	Assinatura digital Ed25519 por transação
Auditoria	Exige acesso privilegiado	Histórico completo e cronológico consultável por todos
Custo e complexidade	Baixos	Maiores: prova de trabalho, validação e replicação

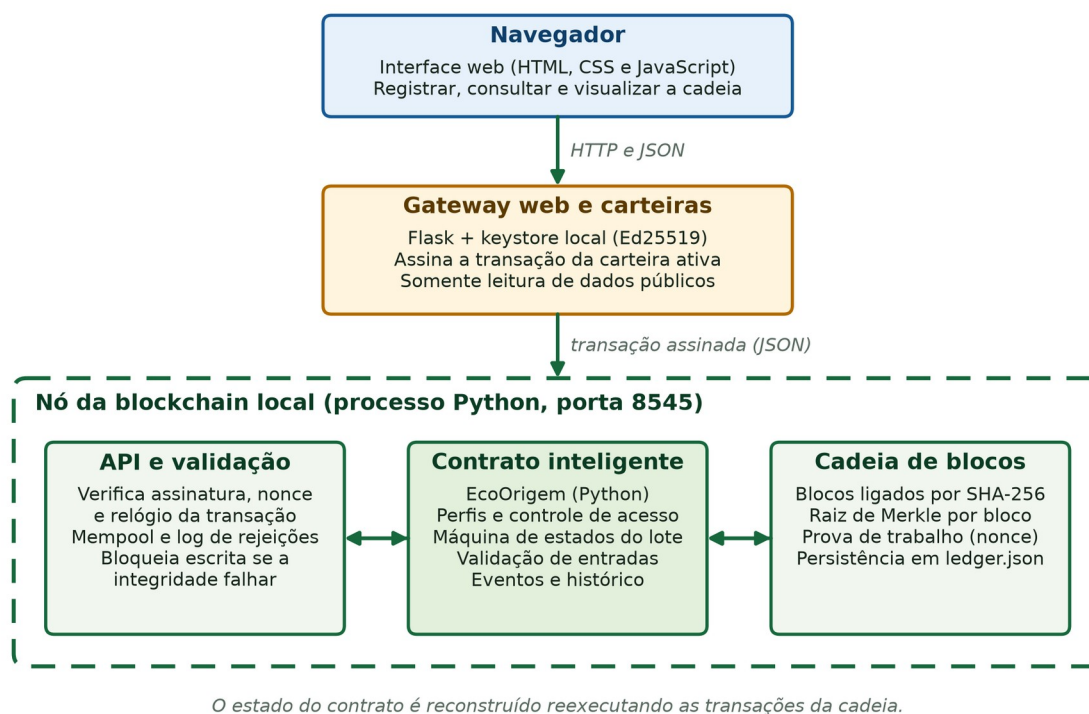
Legenda: a coluna da direita descreve o comportamento do protótipo, com as ressalvas da seção 9.

Blockchain não é a melhor resposta para todo problema. Se uma única organização confiável controlasse toda a cadeia, um banco de dados bastaria. Aqui o valor está no compartilhamento de um registro imutável entre partes que não confiam plenamente umas nas outras. Ainda assim, a blockchain garante que o dado registrado não muda depois de gravado, e não que o dado seja verdadeiro na origem. Essa limitação é discutida na seção 9.

4 Arquitetura

A arquitetura segue o modelo de aplicação descentralizada da Aula 01: interface, contrato inteligente e blockchain, com uma carteira responsável por assinar as transações. A Figura 1 mostra os componentes e a Tabela 2 descreve a responsabilidade de cada um.

Figura 1 - Arquitetura do EcoOrigem



Legenda: o navegador conversa apenas com o gateway web; o gateway assina a transação com a carteira ativa e a envia ao nó, onde o contrato executa as regras e a cadeia registra o resultado.

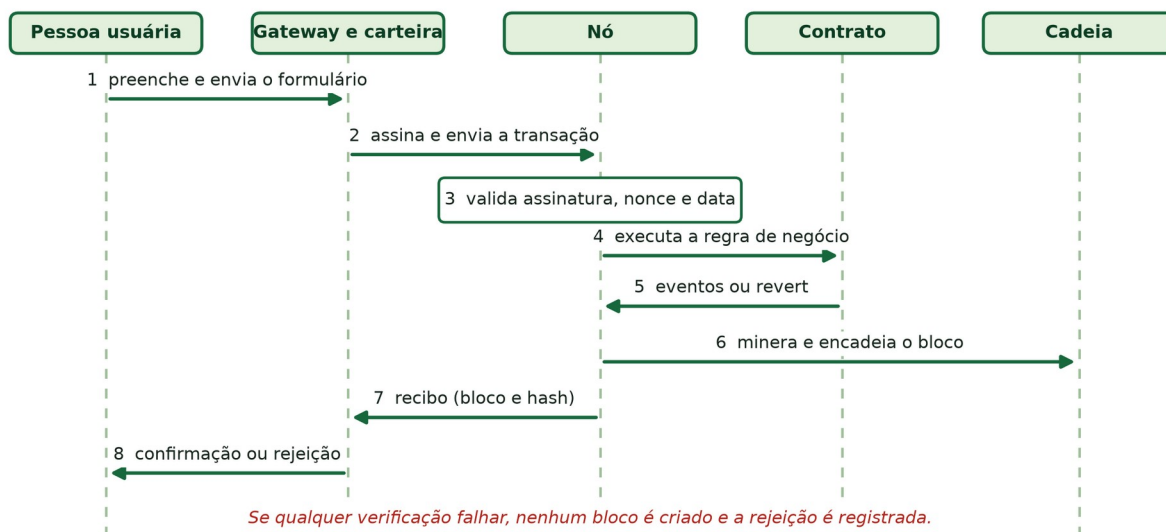
Tabela 2 - Componentes e responsabilidades

Componente	Tecnologia	Responsabilidade
Interface web	HTML, CSS e JavaScript	Formulários, consulta de lotes, explorador de blocos e laboratório de integridade
Gateway e carteiras	Flask, Ed25519	Guarda as chaves de demonstração, assina a transação e encaminha leituras ao nó
Nó da blockchain	Flask, Python	Recebe transações assinadas, valida, executa o contrato, minera e persiste os blocos
Contrato inteligente	Python (EcoOrigemContract)	Perfis, máquina de estados, validações e eventos
Cadeia de blocos	Python (Block, Blockchain)	Encadeamento por SHA-256, prova de trabalho, raiz de Merkle e validação
Empacotamento	Docker, Makefile, venv	Execução reproduzível, testes em contêiner e menu interativo

Legenda: cada linha corresponde a um pacote do repositório, descrito na seção 5.

Uma operação percorre o caminho da Figura 2. O nó só inclui um bloco quando a assinatura, o nonce, a data e todas as regras do contrato passam. Em qualquer falha, nenhum bloco é criado e o motivo retorna à pessoa usuária.

Figura 2 - Ciclo de vida de uma operação



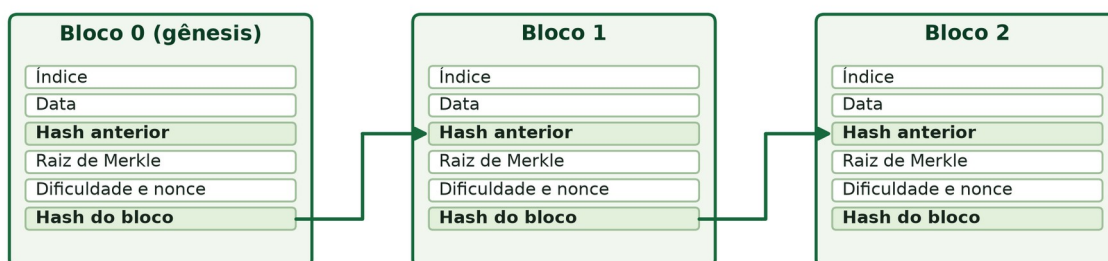
Legenda: as etapas 1 a 8 mostram a passagem da operação pelas camadas até o bloco minerado ou até a rejeição.

5 Implementação

5.1 Blockchain

O bloco reproduz o modelo visto em aula: índice, data, hash anterior, nonce e hash SHA-256. Duas evoluções aproximam o protótipo de uma blockchain real. O cabeçalho passa a guardar a raiz da árvore de Merkle das transações, e cada transação é assinada digitalmente. A Figura 3 mostra o encadeamento.

Figura 3 - Estrutura e encadeamento dos blocos



O hash anterior de cada bloco repete o hash do bloco que o precede. Alterar um bloco quebra todos os seguintes.

Legenda: a seta parte do hash de um bloco e chega ao campo hash anterior do bloco seguinte; alterar qualquer campo muda o hash e rompe essa ligação.

- **Hash e serialização.** O hash do cabeçalho é o SHA-256 de um JSON canônico, com chaves ordenadas e sem espaços, o que produz o mesmo hash em qualquer máquina.
- **Prova de trabalho.** O mineiro incrementa o nonce até o hash começar com a quantidade de zeros definida pela dificuldade, quatro por padrão. Em média são cerca de 65 mil tentativas por bloco, o que leva frações de segundo em Python.
- **Árvore de Merkle.** As transações do bloco são resumidas em uma raiz. Uma prova de inclusão com poucos hashes irmãos demonstra que uma transação pertence ao bloco, e a interface exibe essa prova.
- **Assinatura digital.** Cada transação leva a chave pública, o nonce e a assinatura Ed25519 sobre o seu hash. O endereço da conta é derivado da chave pública, e o nonce por conta impede a repetição de uma transação já aceita.

A validação percorre toda a cadeia e lista cada problema encontrado, em vez de parar no primeiro. A Tabela 3 resume as verificações.

Tabela 3 - Verificações de integridade da cadeia

Código	O que detecta
HASH_MISMATCH	O conteúdo do cabeçalho não corresponde ao hash armazenado
BROKEN_LINK	O hash anterior não coincide com o hash do bloco precedente
POW_INVALID	O hash não atende à dificuldade da prova de trabalho
MERKLE_MISMATCH	As transações não correspondem à raiz de Merkle do bloco
TX_SIGNATURE_INVALID	A assinatura ou o endereço do remetente não conferem
TX_NONCE_INVALID e DUPLICATE_TX	Nonce fora de sequência ou transação repetida
CONTRACT_REPLAY_FAILED	Reexecutar as transações no contrato produz uma rejeição
BAD_INDEX, TIMESTAMP_ORDER, GENESIS_INVALID	Sequência, ordem cronológica e formato do bloco gênese

Legenda: a mesma lista é exibida na aba Integridade da interface quando uma adulteração é simulada.

5.2 Contrato inteligente

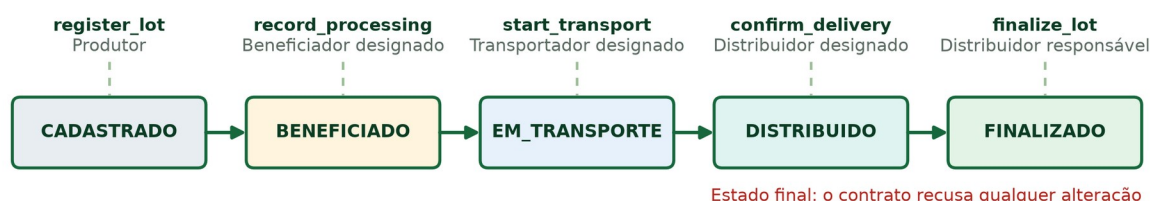
O contrato EcoOrigem é uma classe Python executada pelo nó a cada transação. Seu estado é uma função pura das transações gravadas: reexecutar a cadeia desde o bloco gênese reconstrói exatamente o mesmo estado, e é assim que o nó reinicia. Cada método segue o padrão verificar antes de alterar, usado em Solidity: permissões e entradas são checadas primeiro e o estado só muda depois de todas as verificações passarem. Uma falha equivale a um revert.

Tabela 4 - Perfis de acesso

Perfil	Pode executar	Restrição adicional
Administrador	Conceder e revogar perfis	É a conta que implantou o contrato
Produtor	Registrar lote	Designa o beneficiador do lote
Beneficiador	Registrar beneficiamento	Só o beneficiador designado; designa o transportador
Transportador	Iniciar transporte	Só o transportador designado; designa um distribuidor com perfil válido
Distribuidor	Confirmar recebimento e finalizar	Só o distribuidor designado confirma; só o responsável atual finaliza
Consumidor	Consultar lotes	Leitura pública, sem carteira

Legenda: cada etapa designa quem executa a próxima; o responsável atual pelo lote muda a cada etapa e é o único que pode anexar documentos.

Figura 4 - Ciclo de vida do lote



Saltar ou repetir etapas é rejeitado com INVALID_TRANSITION

Legenda: cada etapa só é aceita depois da anterior e pelo perfil indicado; o estado FINALIZADO é definitivo.

Tabela 5 - Métodos do contrato

Método	Perfil	Efeito	Principais rejeições
grant_role e revoke_role	Administrador	Concede ou revoga um perfil	ACCESS_DENIED, INVALID_INPUT
register_lot	Produtor	Cria o lote em CADASTRADO	Produto fora da lista, quantidade inválida, data futura
record_processing	Beneficiador designado	CADASTRADO para BENEFICIADO	Outro beneficiador tenta registrar, INVALID_TRANSITION
start_transport	Transportador designado	BENEFICIADO para EM_TRANSPORTE	Outro transportador tenta iniciar, destinatário sem perfil de distribuidor
confirm_delivery	Distribuidor designado	EM_TRANSPORTE para DISTRIBUIDO	Outro distribuidor tenta confirmar
finalize_lot	Distribuidor responsável	DISTRIBUIDO para FINALIZADO	INVALID_TRANSITION
attach_document	Responsável atual	Registra o hash de um documento	LOT_FINALIZED, hash repetido ou inválido

Legenda: toda operação sobre lote FINALIZADO retorna LOT_FINALIZED.

As entradas passam por validadores que normalizam e limitam cada campo: produto da lista aceita, com tolerância a caixa e acentos; origem entre 3 e 120 caracteres; quantidade positiva, finita e limitada; data de coleta válida e não futura; hashes de documento com 64 caracteres hexadecimais; campos desconhecidos rejeitados. As mensagens de erro são em português e cada rejeição carrega um código estável.

5.3 Nó da blockchain

- **Aceitação.** O nó verifica assinatura, endereço, nonce esperado, transação repetida e diferença de relógio de no máximo cinco minutos antes de executar o contrato.
- **Mineração.** Com a mineração automática cada transação aceita vira um bloco. No modo manual, as transações ficam na fila e um único bloco as agrupa, o que mostra a árvore de Merkle com várias folhas.
- **Persistência.** A cadeia é gravada de forma atômica em JSON após cada bloco. Ao iniciar, o nó relê o arquivo, valida a cadeia e reconstrói o estado do contrato.
- **Rejeições.** Toda operação recusada é registrada em um log local com camada, código, motivo, carteira e método. Rejeições não criam bloco nem consomem nonce.
- **Bloqueio.** Se a validação de integridade falhar, o nó recusa novas escritas com o código CHAIN_COMPROMISED até a cadeia ser restaurada.

5.4 Interface web

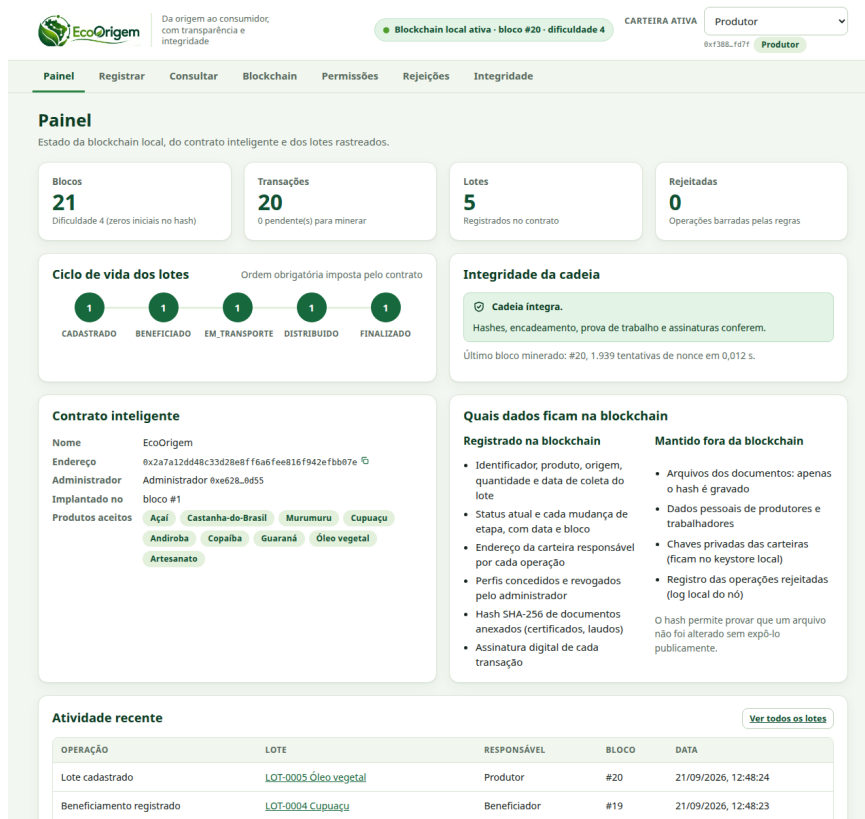
A interface é uma aplicação de página única, sem dependências externas, servida pelo gateway Flask. A carteira ativa fica sempre visível no topo, e o estado da blockchain local é exibido em tempo real.

Tabela 6 - Telas da interface

Tela	Função
Painel	Blocos, transações, lotes por etapa, integridade, contrato e atividade recente
Registrar	Formulários das seis operações, resultado com hash e bloco, cenários de rejeição
Consultar	Lista com filtros, linha do tempo do lote, documentos e verificação de arquivo
Blockchain	Blocos com hash, hash anterior, raiz de Merkle, nonce, transações e prova de inclusão
Permissões	Carteiras, perfis e operações do administrador
Rejeições	Log das operações recusadas, com camada e motivo
Integridade	Simulação de adulteração, validação da cadeia e restauração

Legenda: as capturas de tela de cada uma estão no Apêndice B.

Figura 5 - Painel da aplicação



Legenda: visão geral com os indicadores da cadeia, o ciclo de vida dos lotes e o estado do contrato.

5.5 Infraestrutura

A solução é dockerizada e também roda localmente. O arquivo docker-compose.yml sobe três serviços em ordem: o nó, uma implantação única do contrato com concessão dos perfis de demonstração e a interface. Um perfil opcional cria lotes de exemplo e outro executa os testes em contêiner. O Makefile reúne os comandos principais, cria o ambiente virtual .venv-ecoorigem quando necessário e oferece o comando make menu, um menu interativo com cada opção explicada.

6 Dados na blockchain

A separação entre o que é gravado na cadeia e o que permanece fora dela protege a privacidade e evita crescimento desnecessário. Arquivos e dados pessoais não são gravados. Quando é preciso provar a autenticidade de um documento, grava-se apenas o seu hash SHA-256, e a interface recalcula o hash de um arquivo escolhido e o compara com o registrado.

Tabela 7 - Dados gravados e dados mantidos fora da blockchain

Na blockchain	Fora da blockchain
Identificador, produto, origem, quantidade e data de coleta do lote	Arquivos de certificados, laudos e fotos
Status atual e cada mudança de etapa, com data e bloco	Dados pessoais de produtores e trabalhadores
Endereço da carteira responsável por cada operação	Chaves privadas, mantidas no keystore local
Perfis concedidos e revogados pelo administrador	Log das operações rejeitadas, mantido pelo nó
Hash SHA-256 de documentos anexados	Conteúdo dos documentos
Assinatura digital de cada transação	Senhas ou credenciais de qualquer tipo

Legenda: o hash prova que um arquivo não foi alterado sem expô-lo publicamente.

7 Testes

7.1 Estratégia

Os testes automatizados usam pytest e cobrem, em camadas, a criptografia, a cadeia, o contrato, o nó, a API HTTP, o gateway, o cliente e a linha de comando. Foram executados 251 testes, todos aprovados, com cobertura de 97% das linhas e ramos do pacote. A Tabela 8 resume o resultado das verificações de qualidade.

Tabela 8 - Verificações de qualidade executadas

Verificação	Ferramenta	Resultado
Testes automatizados	pytest	251 aprovados
Cobertura	pytest-cov	97%
Análise estática	Pylint	Nota 10.00 de 10
Formatação	Black	Há arquivos a formatar
Segurança do código	Bandit	Nenhum achado no pacote
Dependências	pip-audit	Nenhuma vulnerabilidade conhecida
Varredura de composição de software	Black Duck	Não executada: ferramenta comercial não disponível neste ambiente

Legenda: o resultado da varredura Black Duck deve ser registrado pela equipe quando a ferramenta estiver disponível.

Tabela 9 - Testes automatizados por área

Área coberta	Testes
Blocos, prova de trabalho e validação	25
Linha de comando e cliente HTTP	11
Contrato: perfis, etapas e entradas	87
test_custody.py	3
Hash e árvore de Merkle	16
Nó: implantação, replay, persistência e laboratório	40
API HTTP do nó	15
Transações assinadas	25
Carteiras, assinaturas e keystore	11
Gateway web e carteiras	18
Total	251

Legenda: parametrizações contam como testes individuais.

7.2 Cenários de operações válidas, inválidas e de integridade

Além dos testes de unidade, 48 cenários foram executados contra uma blockchain nova, com transações reais assinadas, regras do contrato e mineração. Cada cenário declara o resultado esperado e o compara com o obtido. O resultado foi 48 de 48 conforme o esperado. O Apêndice A traz a tabela completa e o comando `make scenarios` a reproduz.

Tabela 10 - Resumo dos cenários executados

Grupo	Cenários	Exemplos
Operações válidas	16	Implantar, conceder perfis, percorrer as cinco etapas, anexar e verificar documento, reiniciar o nó
Entradas inválidas ou sem permissão	27	Quantidade negativa, data futura, pular etapas, alterar lote finalizado, assinatura falsa, repetição de transação
Integridade	5	Alterar dado gravado, refazer a prova de trabalho, bloquear escrita e restaurar

Legenda: os identificadores V, I e A correspondem às linhas do Apêndice A.

7.3 Resultados obtidos

Todas as operações válidas foram confirmadas em novos blocos e o histórico completo do lote pôde ser consultado. Todas as entradas inválidas e sem permissão foram rejeitadas com o código esperado, sem criar bloco e sem alterar o estado do contrato. A adulteração de um dado gravado foi detectada pela validação, tanto sem recalcular hashes, por divergência da raiz de Merkle e da assinatura, quanto refazendo a prova de trabalho do bloco, por quebra do encadeamento com o bloco seguinte. Com a cadeia comprometida, o nó recusou novas operações até a restauração.

8 Como executar

Os passos abaixo iniciam a blockchain local, implantam o contrato e executam a aplicação. Os pré-requisitos são Docker com Compose ou Python 3.10 ou superior com make.

8.1 Com Docker

```
git clone https://github.com/JulianaBallin/EcoOrigem.git
cd EcoOrigem
make docker-up      # nó, implantação do contrato e interface
make docker-seed    # opcional: lotes de demonstração
# Interface: http://127.0.0.1:5000    Nó: http://127.0.0.1:8545
```

8.2 Sem Docker

```
make venv           # cria .venv-ecoorigem e instala as dependências
make node           # terminal 1: blockchain local (porta 8545)
make deploy         # terminal 2: implanta o contrato e concede perfis
make app            # terminal 3: interface (porta 5000)
make start          # alternativa: faz tudo acima em segundo plano
```

Tabela 11 - Principais comandos do Makefile

Comando	Função
make menu	Menu interativo com todas as opções explicadas
make test, make coverage	Testes automatizados, com ou sem cobertura
make lint, make format, make security	Pylint, Black e Bandit com pip-audit
make scenarios	Executa os cenários e gera a tabela de resultados
make docker-test	Executa os testes dentro de um contêiner
make docker-reset, make reset	Apagam a blockchain e as carteiras para recomençar

9 Limitações

- **Nó único.** A blockchain roda em um processo local. Sem uma rede de nós que rejeitem a cadeia adulterada, quem controla o arquivo pode refazer todos os blocos. A validação detecta a alteração, mas a imutabilidade plena exige replicação entre participantes.
- **Prova de trabalho didática.** A dificuldade é baixa e serve para demonstrar o mecanismo, sem a segurança econômica de uma rede pública.
- **Carteiras no servidor.** As chaves de demonstração ficam em um keystore no gateway, e não em uma extensão como o MetaMask. Isso simplifica a demonstração, mas não deve ser usado em produção.
- **Veracidade na origem.** A blockchain garante que o registro não muda depois de gravado, não que o produtor informou a verdade. Certificadoras e sensores poderiam reforçar esse primeiro registro.

- **Dados públicos.** Tudo que é gravado é legível por quem acessa a cadeia. Por isso dados pessoais ficam fora e documentos entram apenas como hash.
- **Escala.** A reconstrução do estado reexecuta a cadeia inteira e o armazenamento é um arquivo JSON, adequados a um protótipo.
- **Implantação aberta.** Qualquer conta pode implantar o contrato primeiro. Em produção a conta implantadora deveria ser restrita.
- **Black Duck.** A varredura de composição de software não foi executada por indisponibilidade da ferramenta.

10 Conclusão

O EcoOrigem mostra, de ponta a ponta, como uma blockchain local em Python pode sustentar a rastreabilidade de produtos da bioeconomia amazônica. O contrato inteligente impõe a ordem das etapas e os perfis de acesso, a cadeia torna o histórico verificável e a interface permite registrar, consultar e auditar sem conhecimento técnico. Os testes comprovam tanto o funcionamento esperado quanto a rejeição de entradas inválidas e a detecção de adulterações.

Como evolução, a equipe propõe: replicar a cadeia entre vários nós com consenso; integrar uma carteira externa como o MetaMask, com assinatura no navegador; gerar QR Code por lote para consulta pública pelo consumidor; registrar a localização aproximada da coleta; e integrar certificadoras e sensores para fortalecer a veracidade do primeiro registro.

Referências

- NAKAMOTO, S. Bitcoin: a peer-to-peer electronic cash system. 2008. Disponível em: <https://bitcoin.org/bitcoin.pdf>.
- SANTOS, F. Blockchain: visão geral. Material da Aula 01, Oficina de Desenvolvimento de Sistemas III, UEA, 2026.
- SANTOS, F. Implementação de Blockchain. Material da Aula 02 e exemplos BlockchainExemplo e Blockchain_APP, UEA, 2026.
- NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. FIPS 180-4: Secure Hash Standard. 2015.
- JOSEFSSON, S.; LIUSVAARA, I. RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA). IETF, 2017.
- MERKLE, R. C. A digital signature based on a conventional encryption function. CRYPTO 1987.

Apêndice A - Cenários de teste

Operações realizadas, resultados esperados e resultados obtidos na execução automatizada dos cenários. Os números de bloco variam a cada execução.

Tabela 12 - Cenários executados

ID	Operação realizada	Resultado esperado	Resultado obtido	Status
V01	Implantar o contrato com a carteira Administrador	confirmada em um novo bloco	confirmada no bloco 1	Aprovado
V02	Administrador concede o perfil PRODUCER a produtor	confirmada em um novo bloco	confirmada no bloco 2	Aprovado
V03	Administrador concede o perfil PROCESSOR a beneficiador	confirmada em um novo bloco	confirmada no bloco 3	Aprovado
V04	Administrador concede o perfil CARRIER a transportador	confirmada em um novo bloco	confirmada no bloco 4	Aprovado
V05	Administrador concede o perfil DISTRIBUTOR a distribuidor	confirmada em um novo bloco	confirmada no bloco 5	Aprovado
V06	Administrador concede o perfil DISTRIBUTOR a distribuidor2	confirmada em um novo bloco	confirmada no bloco 6	Aprovado
V07	Produtor registra o lote LOT-0001 com hash do documento de origem	confirmada em um novo bloco	confirmada no bloco 7	Aprovado
V08	Beneficiador registra o beneficiamento de LOT-0001	confirmada em um novo bloco	confirmada no bloco 8	Aprovado
V09	Transportador inicia o transporte designando o distribuidor	confirmada em um novo bloco	confirmada no bloco 9	Aprovado
V10	Distribuidor designado confirma o recebimento	confirmada em um novo bloco	confirmada no bloco 10	Aprovado
V11	Distribuidor finaliza o lote	confirmada em um novo bloco	confirmada no bloco 11	Aprovado
V12	Consultar LOT-0001: status e histórico completo	FINALIZED com 5 eventos	FINALIZED com 5 eventos	Aprovado
V13	Responsável anexa documento ao lote LOT-0002	confirmada em um novo bloco	confirmada no bloco 13	Aprovado
V14	Verificar documento com hash correto	registrado	registrado	Aprovado
V15	Validar a cadeia inteira	íntegra	íntegra	Aprovado
V16	Reiniciar o nó e reconstruir o estado do disco	2 lotes, cadeia íntegra	2 lotes, cadeia íntegra	Aprovado
I01	Carteira sem perfil tenta registrar lote	rejeitada (ACCESS_DENIED)	rejeitada (ACCESS_DENIED)	Aprovado
I02	Quantidade negativa	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I03	Quantidade zero	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I04	Data de coleta no futuro	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I05	Produto fora da lista aceita	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I06	Origem vazia	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I07	Campo desconhecido no cadastro	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I08	Finalizar lote CADASTRADO (pular etapas)	rejeitada (INVALID_TRANSITION)	rejeitada (INVALID_TRANSITION)	Aprovado

ID	Operação realizada	Resultado esperado	Resultado obtido	Status
I09	Repetir o beneficiamento de lote já beneficiado	rejeitada (INVALID_TRANSITION)	rejeitada (INVALID_TRANSITION)	Aprovado
I10	Beneficiar lote inexistente	rejeitada (LOT_NOT_FOUND)	rejeitada (LOT_NOT_FOUND)	Aprovado
I11	Identificador de lote em formato inválido	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I12	Alterar lote FINALIZADO (reprocessar)	rejeitada (LOT_FINALIZED)	rejeitada (LOT_FINALIZED)	Aprovado
I13	Anexar documento a lote FINALIZADO	rejeitada (LOT_FINALIZED)	rejeitada (LOT_FINALIZED)	Aprovado
I14	Produtor tenta conceder perfil (não é administrador)	rejeitada (ACCESS_DENIED)	rejeitada (ACCESS_DENIED)	Aprovado
I15	Conceder perfil já existente	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I16	Destinatário do transporte sem perfil Distribuidor	rejeitada (INVALID_INPUT)	rejeitada (INVALID_INPUT)	Aprovado
I17	Método inexistente no contrato	rejeitada (UNKNOWN_METHOD)	rejeitada (UNKNOWN_METHOD)	Aprovado
I18	Anexar documento sem ser o responsável atual	rejeitada (ACCESS_DENIED)	rejeitada (ACCESS_DENIED)	Aprovado
I19	Nenhum bloco foi criado pelas 18 operações rejeitadas	0 blocos novos	0 blocos novos	Aprovado
I20	Outro distribuidor tenta confirmar o recebimento	rejeitada (ACCESS_DENIED)	rejeitada (ACCESS_DENIED)	Aprovado
I21	Transação com remetente falsificado	rejeitada (INVALID_SIGNATURE)	rejeitada (INVALID_SIGNATURE)	Aprovado
I22	Transação alterada depois de assinada	rejeitada (INVALID_SIGNATURE)	rejeitada (INVALID_SIGNATURE)	Aprovado
I23	Reenvio da mesma transação (replay)	rejeitada (DUPLICATE_TRANSACTION)	rejeitada (DUPLICATE_TRANSACTION)	Aprovado
I24	Nonce fora de sequência	rejeitada (NONCE_MISMATCH)	rejeitada (NONCE_MISMATCH)	Aprovado
I25	Transação com data muito antiga	rejeitada (TIMESTAMP_OUT_OF_RANGE)	rejeitada (TIMESTAMP_OUT_OF_RANGE)	Aprovado
I26	Reimplantar o contrato	rejeitada (CONTRACT_ALREADY_DEPLOYED)	rejeitada (CONTRACT_ALREADY_DEPLOYED)	Aprovado
I27	Verificar arquivo alterado (hash diferente)	não registrado	não registrado	Aprovado
A01	Alterar dado gravado no bloco 18 (sem refazer hashes)	inválida a partir do bloco 18	inválida a partir do bloco 18	Aprovado
A02	Nova operação com a cadeia comprometida	rejeitada (CHAIN_COMPROMISED)	rejeitada (CHAIN_COMPROMISED)	Aprovado
A03	Alterar o bloco 17 e refazer a prova de trabalho dele	encadeamento quebrado (BROKEN_LINK)	encadeamento quebrado (BROKEN_LINK)	Aprovado
A04	Restaurar a cadeia a partir do disco	íntegra	íntegra	Aprovado
A05	Operação volta a ser aceita após a restauração	confirmada em um novo bloco	confirmada no bloco 19	Aprovado

Legenda: Aprovado indica que o resultado obtido coincide com o esperado.

Apêndice B - Evidências da interface

Capturas de tela geradas automaticamente pelo roteiro de demonstração (make demo-video), executado sobre a stack Docker.

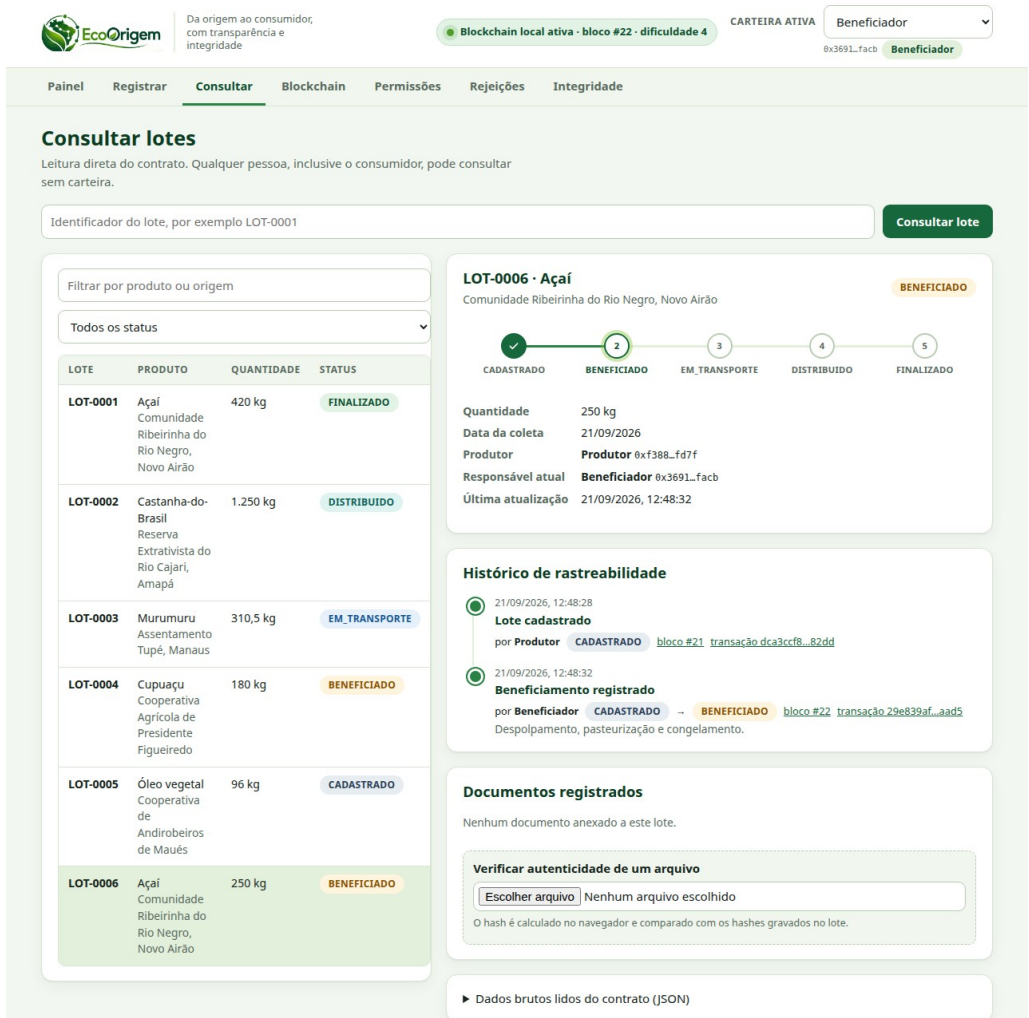
Figura 6 - Formulário de registro de lote

Legenda: formulário preenchido com a carteira Produtor antes do envio.

Figura 7 - Operação confirmada

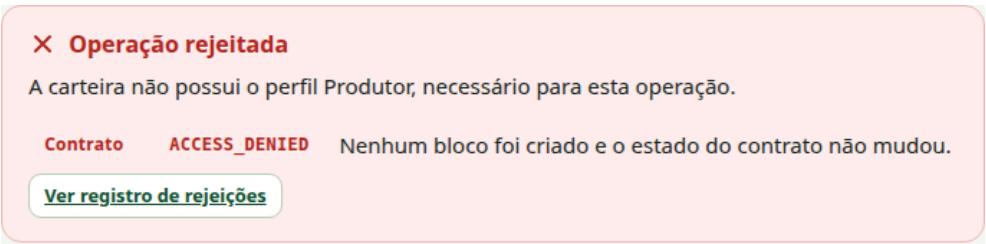
Legenda: resultado com hash da transação, bloco minerado e prova de trabalho.

Figura 8 - Consulta e histórico do lote



Legenda: etapas, responsáveis e linha do tempo com bloco e transação de cada evento.

Figura 9 - Rejeição por falta de permissão



Legenda: carteira sem perfil tenta registrar lote e o contrato retorna ACCESS_DENIED.

Figura 10 - Rejeição por etapa fora de ordem

×

Operação rejeitada

Transição inválida: CADASTRADO para FINALIZADO. A próxima etapa permitida é BENEFICIADO.

Contrato

INVALID_TRANSITION

Nenhum bloco foi criado e o estado do contrato não mudou.

Ver registro de rejeições

Legenda: tentativa de finalizar um lote apenas cadastrado.

Figura 11 - Log de operações rejeitadas

EcoOrigem

Da origem ao consumidor, com transparência e integridade

Blockchain local ativa · bloco #22 · dificuldade 4

CARTEIRA ATIVA

Beneficiador

6x3691...factb

Beneficiador

Painel

Registrar

Consultar

Blockchain

Permissões

Rejeições

Integridade

Operações rejeitadas

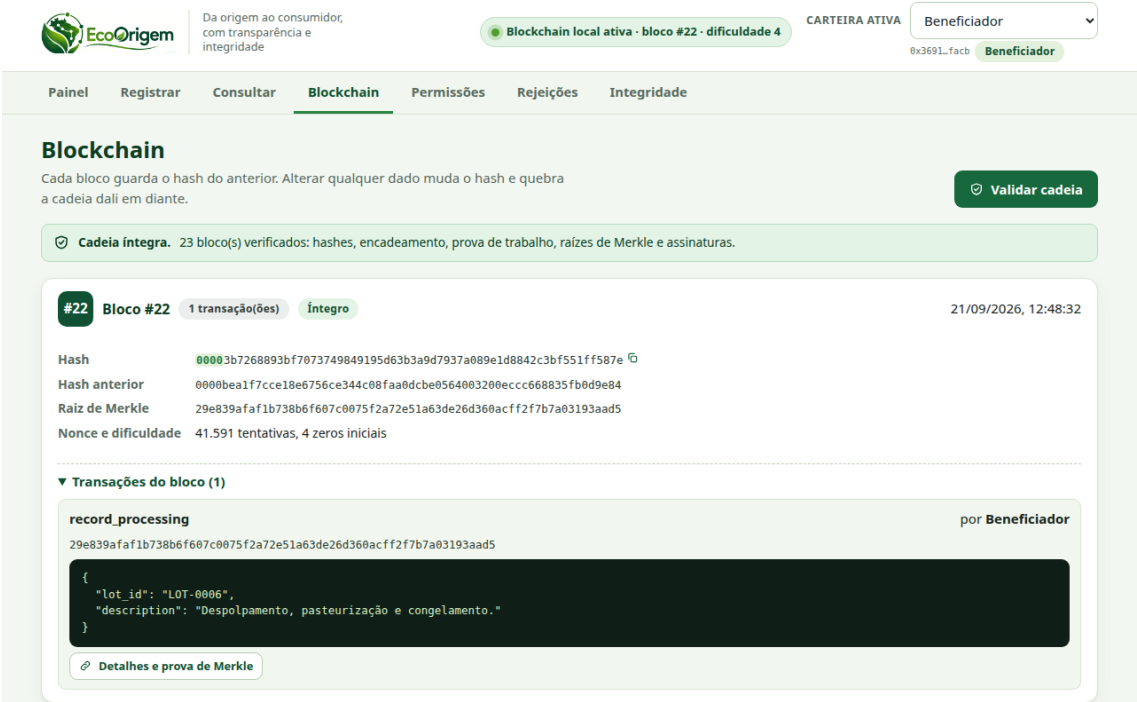
Toda tentativa barrada pelas regras do contrato ou pela validação do nó fica registrada aqui. Rejeições não geram bloco e não alteram o estado.

DATA	CAMADA	CÓDIGO	MOTIVO	CARTEIRA	MÉTODO
21/09/2026, 12:48:47	Contrato	LOT_FINALIZED	O lote LOT-0001 está FINALIZADO e não aceita novas alterações.	Beneficiador	record_processing
21/09/2026, 12:48:45	Contrato	INVALID_TRANSITION	Transição inválida: CADASTRADO para FINALIZADO. A próxima etapa permitida é BENEFICIADO.	Distribuidor	finalize_lot
21/09/2026, 12:48:42	Contrato	INVALID_INPUT	O campo 'quantity_kg' deve ser maior que zero.	Produtor	register_lot
21/09/2026, 12:48:39	Contrato	ACCESS_DENIED	A carteira não possui o perfil Produtor, necessário para esta operação.	Carteira sem permissão	register_lot

EcoOrigem 1.0.0 Protótipo acadêmico. Blockchain local em Python, sem valor monetário. Oficina de Desenvolvimento de Sistemas III, UEA

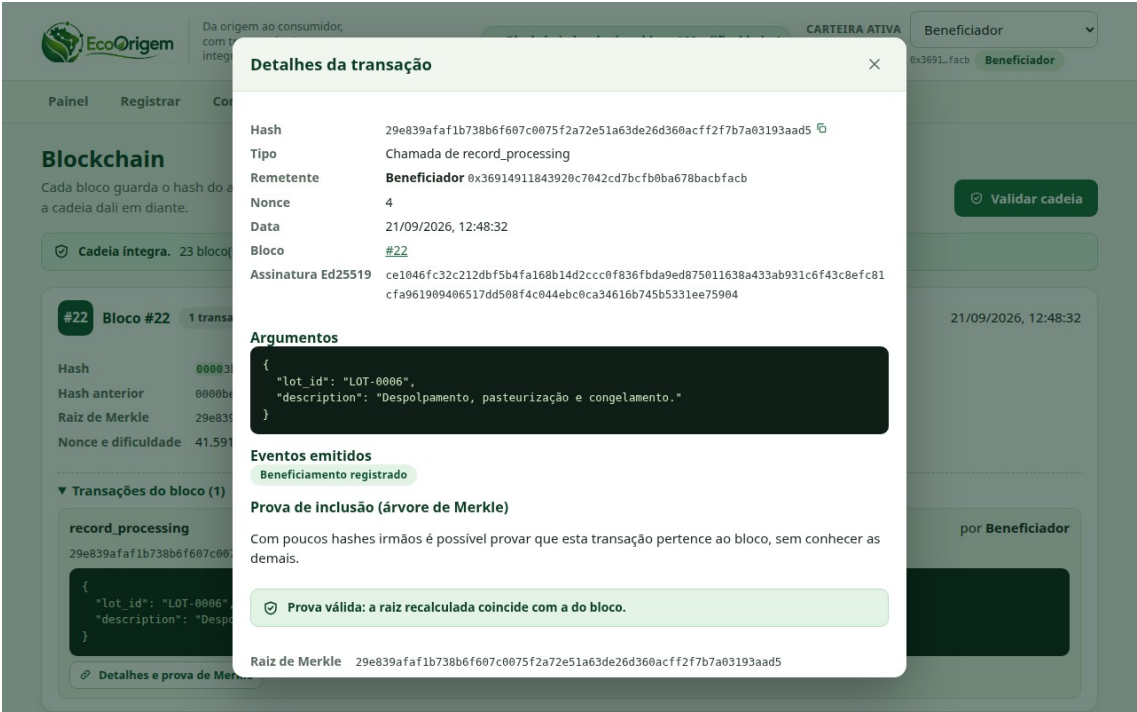
Legenda: todas as rejeições com camada, código, motivo, carteira e método.

Figura 12 - Explorador de blocos



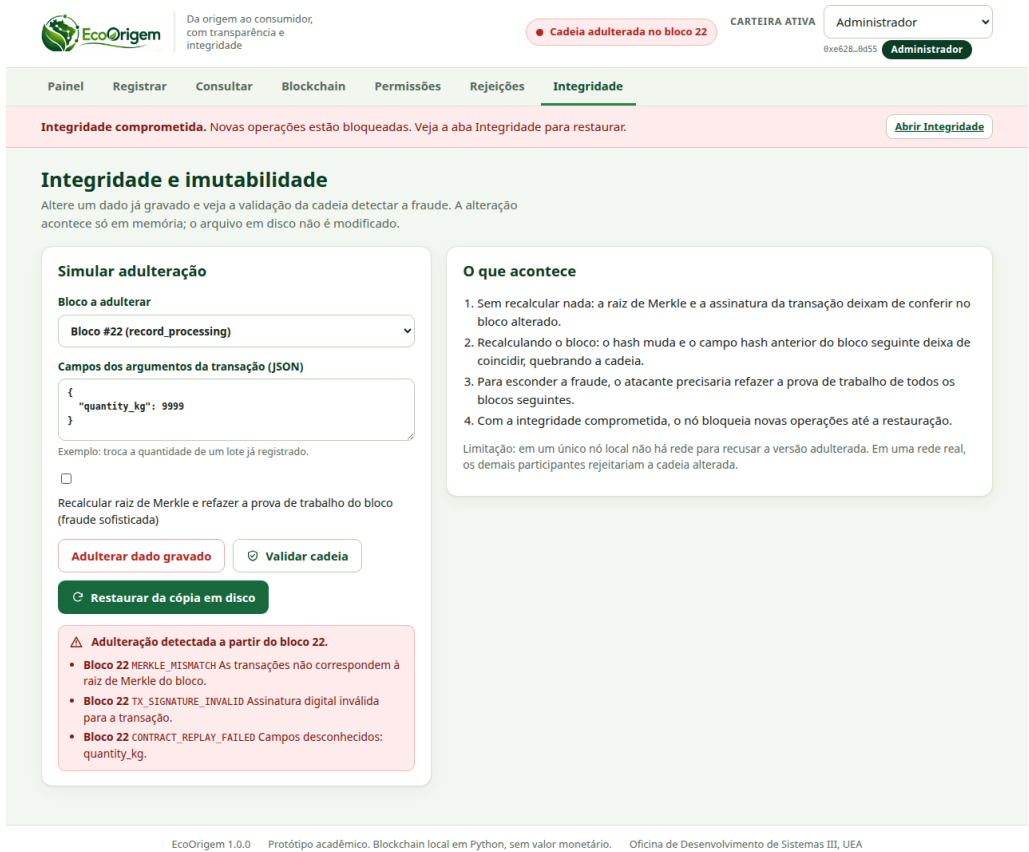
Legenda: hash com zeros da prova de trabalho, hash anterior, raiz de Merkle e transações.

Figura 13 - Prova de inclusão de Merkle



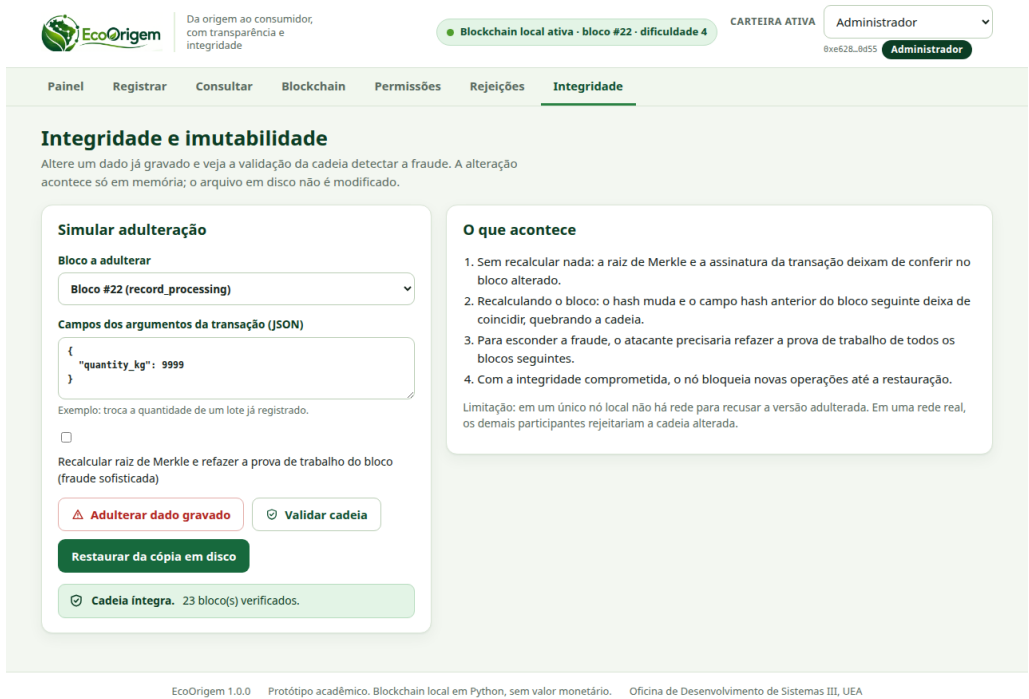
Legenda: detalhe da transação com a prova validada contra a raiz do bloco.

Figura 14 - Adulteração detectada



Legenda: a alteração de um dado gravado invalida o bloco e bloqueia novas operações.

Figura 15 - Cadeia restaurada



Legenda: após a restauração a validação volta a indicar integridade.